AEGIS Learn: A Retrieval-Augmented Generative AI Framework for Personalized, Bloom's-Taxonomy-Aligned Learning

Masuma Fathema Avinash Yarukala Madhav Ghattamaneni K V S Jaswanth

Supervisor – Dr. K. Swanthana

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India

***Abstract*—Static textbooks and conventional e-learning systems present largely uniform material to learners who differ in prior knowledge, pace, and cognitive readiness. This paper presents AEGIS Learn, a retrieval-augmented generative AI framework for personalized undergraduate learning. The framework indexes reference material from OpenStax, NCERT, and a Khan Academy algebra corpus, retrieves semantically relevant passages using vector similarity, and combines the retrieved context with a Bloom's Taxonomy cognitive-level tag in a structured prompt for Gemini 1.5 Flash. A React, TypeScript, and Vite interface exposes student, educator, and administrator surfaces, while performance tracking is designed to record mastery across Bloom's levels and support remedial recommendations. The implemented prototype integrates the user interface, dataset processing, similarity search, Bloom's Taxonomy-based prompt templates, Gemini integration, authentication, quiz generation, and learner performance tracking. Further empirical evaluation is required to assess retrieval quality, generation reliability, and educational effectiveness. The design addresses three recurring challenges in educational GenAI: factual reliability, curriculum alignment, and insufficiently structured assessment.**

***Index Terms*—Personalized learning, retrieval-augmented generation, Bloom's Taxonomy, large language models, vector similarity search, educational technology.**

I. Introduction

Textbooks and static course material are commonly written for an average learner, yet real classrooms contain students with different prior knowledge, learning pace, preferred approaches, and cognitive readiness. Bloom's Taxonomy organizes learning objectives into six cognitive levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating [1], [2]. Although instructors use this hierarchy to plan learning outcomes, manually producing level-appropriate notes and assessments for every topic and learner does not scale.

Large language models can generate explanations, examples, and questions on demand. However, a model answering primarily from parametric memory can drift from the textbook or syllabus being used and may produce fluent but unsupported information. Retrieval-Augmented Generation (RAG) addresses part of this reliability problem by supplying retrieved evidence to the generator at inference time. Research on RAG shows that combining parametric generation with non-parametric retrieved knowledge can improve factuality and knowledge-intensive generation [3], while educational studies highlight both the opportunities and risks of LLM-based tutoring [4].

AEGIS Learn combines textbook-grounded retrieval with Bloom's-Taxonomy-aware prompting to generate personalized study guides, quizzes, and remedial recommendations. The design uses a retrieval-first workflow so that generated content is tied to a curated corpus, while Bloom's Taxonomy is used before generation rather than merely as a post-hoc label.

A. Background

Three research strands meet in AEGIS Learn. The first is adaptive and personalized learning, where a learner model is used to select or sequence content [5]. The second is neural text representation and dense retrieval, where sentence embeddings enable semantic matching and vector indexes make retrieval practical [6], [7], [8]. The third is cognitive assessment using Bloom's Taxonomy. Recent research additionally shows that LLMs can support question generation, explanation generation, automatic assessment, and tutoring, but educational use benefits from explicit pedagogical constraints and reliable source grounding.

B. Problem Statement

- Content is often uniform across learners with different levels, speeds, and learning needs.
- Ungrounded generative AI can hallucinate or provide inaccurate information.
- Many AI learning assistants are not explicitly tied to the learner's assigned curriculum.
- Coding practice, explanations, notes, and assessment are rarely integrated into one learning workflow.
- Assessment systems do not consistently target higher-order cognitive skills.

C. Research Gap

Existing platforms generally address one requirement at a time. Adaptive systems personalize selection but commonly depend on authored content; general-purpose LLM assistants generate content but are not necessarily grounded in a syllabus; RAG assistants improve grounding but do not inherently model learner mastery; and quiz generators can vary difficulty without systematically using Bloom's levels as generation-time controls. The reviewed literature supports these individual directions, while AEGIS Learn combines learner-aware generation, retrieval grounding, Bloom-aligned assessment, and mastery-oriented remediation in one workflow.

D. Research Objectives

- Design a personalized learning framework driven by Generative AI.
- Generate customized study notes from a learner profile and selected topic.
- Provide AI-generated coding examples with explanations.
- Generate quizzes whose cognitive demand follows the learner's requested level.

- Integrate Bloom's Taxonomy into assessment generation and tracking.
- Define a measurable protocol for grounding, relevance, retrieval, personalization, assessment quality, and usability.

E. Literature Foundations and Design Rationale

Personalized learning has a longer research history than today's generative AI tools. Adaptive learning systems and intelligent tutoring environments use evidence from student activity to decide which content, exercises, or explanations may be useful next. Reviews of artificial intelligence in education describe adaptive instruction, tutoring, assessment, and learner analytics as connected but distinct applications [5], [20], [22]. A learner model is most useful when it changes as the learner works, rather than assigning a permanent ability label from a small number of responses [21]. Surveys of personalized learning systems also highlight the relationship between learner models, learning content, and the instructional environment [25]. These ideas inform AEGIS Learn's use of relevant learner-profile information and performance records to shape the next learning activity. The intended support remains complementary to instructor judgment and approved course material [28], [29].

The framework also treats learning as an active process. Constructivist learning environments emphasize meaningful tasks through which learners build explanations, test ideas, and reflect on outcomes [30]. Accordingly, AEGIS Learn is intended to do more than return a ready-made answer. Its study guides and quizzes are designed to help students recall concepts, explain them, apply methods, analyze mistakes, justify choices, and create solutions. Bloom's original taxonomy and its revised form provide the vocabulary for describing these different forms of cognitive work [1], [2]. The selected level is therefore used to shape the task before generation, while the actual cognitive demand still needs to be checked by reviewers.

The retrieval pipeline is grounded in research on contextual text representations and passage matching. Transformer architectures provide the foundation for many modern language representations [27], while BERT introduced bidirectional contextual encoding for language understanding [26]. Sentence-BERT produces sentence embeddings that can be compared efficiently [6], and dense passage retrieval learns compatible representations for questions and candidate passages [8]. Work on large-scale similarity search provides methods for efficient vector lookup [7]. Together, these studies explain the technical basis for representing learner queries and educational passages in a shared vector space. They also support the use of similarity ranking as a practical retrieval mechanism, while reminding us that a high similarity score does not, by itself, prove that a passage fully answers a question.

RAG research has expanded beyond the original retrieve-then-generate pattern to examine how evidence is selected, combined, and supplied to a language model [3], [23], [24]. Educational surveys describe applications of retrieval-augmented systems in tutoring, question answering, content development, and assessment, and identify recurring concerns involving corpus coverage, hallucination, evaluation, computational cost, and response quality [18], [19]. An education-focused comparison in MOOCs provides a relevant example of evaluating retrieval-augmented generation in a learning context [9]. Reviews of LLM-based virtual tutors likewise describe possible uses in explanation, feedback, question generation, and problem solving, while emphasizing the need for human oversight [10]. A 2026 evidence review focused on K-12 settings cautions that immediate task performance should not automatically be interpreted as durable learning or transfer; its findings are therefore a useful caution, not direct evidence about undergraduate outcomes [11].

Research on assessment generation adds a further reason to make pedagogy explicit. Studies have examined question design guided by Bloom's Taxonomy [12], the usefulness of questions generated by large language models [13], automatic question-answer pair generation in higher education [14], and the role of plausible distractors in multiple-choice questions [15]. Work on generative AI for course-content development also illustrates the value of structured prompts and iterative review [16]. These findings support evaluating generated items on several dimensions, including correctness, clarity, answerability, cognitive alignment, and distractor quality, rather than treating fluent wording as sufficient evidence of quality.

Responsible use is part of the framework's educational context. UNESCO guidance for generative AI in education and research emphasizes human-centered adoption, appropriate oversight, privacy, and the need to preserve the role of educators [28]. In AEGIS Learn, these principles translate into practical design expectations: retrieved passages should retain source information, learner records should be accessible only to authorized roles, and generated practice material should not be presented as institutionally approved assessment without review. The selected generator, Gemini 1.5 Flash [17], is an implementation choice for the current prototype; it is not assumed to be the only suitable model for this workflow.

Taken together, the literature motivates a learning workflow that connects curriculum-grounded retrieval, learner-aware generation, explicit cognitive-level control, and follow-up based on performance. The sources also show why these functions should be evaluated separately: retrieval relevance, response faithfulness, pedagogical alignment, learner experience, and learning outcomes are related, but one measure cannot stand in for all the others [18]–[22], [28]–[30]. AEGIS Learn is presented as a prototype framework for investigating this combination, with empirical learning gains reserved for future evaluation.

F. Contributions

- A retrieval-augmented pipeline that grounds generated study notes and quizzes in a curated educational corpus.
- A Bloom's-Taxonomy routing layer that maps a learner request to one of six cognitive levels before generation.
- A three-role web application that separates student learning, educator corpus review, and administrator functions.
- A performance-tracking design that records mastery by Bloom level and supports a remedial learning roadmap.
- An evaluation protocol that distinguishes implemented prototype functions from outcomes requiring controlled validation.

G. Organization of the Paper

Section II reviews related work in retrieval-augmented generation, LLM-based tutoring, Bloom-aligned assessment, and automatic question generation. Section III describes the proposed AEGIS Learn framework and its retrieval and prompt-generation workflow. Section IV presents the datasets, technology stack, and implementation scope. Section V outlines the evaluation plan and an illustrative learning scenario. Section VI discusses limitations and threats to validity, and Section VII concludes the paper and identifies future work.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. introduced Retrieval-Augmented Generation by combining a pretrained generator with a non-parametric memory accessed through retrieval [3]. Their work showed how retrieved passages can support knowledge-intensive generation with more specific evidence. In education, Miladi et al. examined retrieval-augmented GPT models in MOOCs and reported improved response accuracy relative to non-retrieval variants [9]. These studies motivate the retrieval-first design used in AEGIS Learn, where course material is selected before the language model prepares a response.

B. LLM-Based Tutoring and Educational Generation

A review of large language models as virtual tutors identifies applications such as question generation, answer grading, explanation generation, problem solving, summarization, and code correction [10]. These capabilities can make learning support more responsive, but the review also highlights the importance of human oversight and suitable instructional design. A 2026 review of AI evidence in K-12 education similarly cautions that immediate performance gains do not necessarily establish transfer to independent work [11]. Although the educational setting differs from undergraduate learning, this evidence reinforces the need to evaluate AEGIS Learn with real learners rather than infer effectiveness from generated outputs alone.

C. Bloom's Taxonomy in Assessment

Bloom's Taxonomy provides a structured vocabulary for describing cognitive demand. Research on question design guided by Bloom's categories explores ways to make assessment tasks require more than simple recall [12]. Other work reports that questions generated with explicit Bloom-level and difficulty controls can be useful to teachers, while also requiring review for the reasoning they actually demand [13]. AEGIS Learn uses the selected cognitive level as a generation-time instruction and treats alignment as an evaluation question, not as an automatic property of a prompt label.

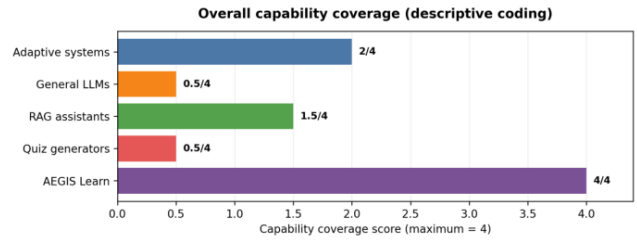


Fig. 2. Descriptive capability coverage score; partial coverage contributes 0.5 and full coverage contributes 1.

The figures indicate that AEGIS Learn is designed to combine all four capability dimensions in one workflow. The comparison should not be interpreted as proof that the system outperforms the listed approaches. The systems differ in purpose, implementation, and evaluation context; controlled benchmarking would be required for performance claims.

D. Automatic Question and MCQ Generation

Research on automatic question-answer pair generation in higher education examines generation pipelines and evaluates outputs through automated measures and educator review [14]. Multiple-choice question generation also depends on the quality of distractors: weak or obviously incorrect options reduce the value of an assessment, while plausible alternatives can help reveal misunderstandings [15]. These findings support reviewing quiz items for answer correctness, clarity, answerability, cognitive alignment, and distractor quality.

E. Generative AI for Course Content Development

The GAIDE framework describes a structured approach to using generative AI in instructional content development, with attention to contextual and pedagogical quality [16]. Its emphasis on explicit prompts and iterative refinement aligns with the AEGIS Learn design choice to maintain separate templates for study guides, quizzes, coding support, and recommendations.

F. Research Gap and Comparative Analysis

TABLE I
QUALITATIVE CAPABILITY COMPARISON BY CATEGORY

Category	Profile	Grounding	Bloom	Mastery
Adaptive systems	Yes	Authored	No	Yes
General LLMs	Partial	No	No	No
RAG assistants	Partial	Yes	No	No
Quiz generators	No	Partial	No	No
AEGIS Learn	Yes	Yes	Yes	Implemented

The comparison is qualitative and describes design coverage only. "Yes" is coded as 1, "Partial" as 0.5, and "No" as 0. The "Authored" grounding category denotes content selected or written in advance rather than passages retrieved for each learner request. These scores do not represent controlled experimental performance.

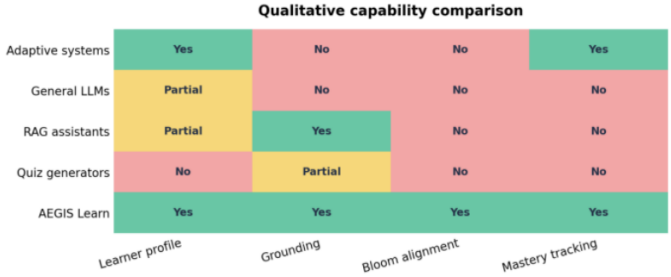


Fig. 1. Capability matrix derived from the qualitative comparison in Table I.

III. Proposed Framework

A. Overview

AEGIS Learn follows a retrieval-grounded workflow: the learner authenticates, selects or accesses a curriculum corpus, submits a topic or learning request, and the system embeds the request into the same vector space as indexed passages. The top-k passages are retrieved by cosine similarity. The request is then routed to a Bloom's Taxonomy level, after which a structured prompt containing the learner profile, retrieved context, task, and cognitive-level tag is sent to Gemini 1.5 Flash [17]. The generated response supports personalized notes, quizzes, coding assistance, and downstream mastery analysis (Fig. 3).

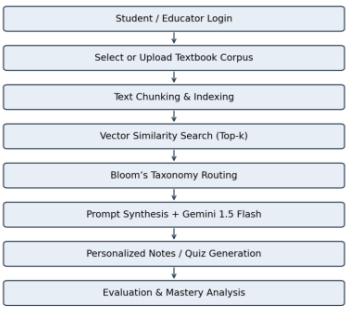


Fig. 3. AEGIS Learn system workflow.

B. Bloom's Taxonomy Routing

The framework uses the six revised cognitive levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating (Fig. 4). A Remembering request can produce definitions and recall items, whereas a Creating request can generate synthesis or design tasks from the same retrieved evidence. Routing the cognitive level before generation makes the taxonomy an active control signal rather than a label attached after generation.

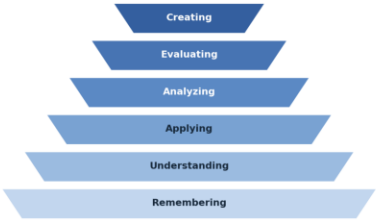


Fig. 4. Bloom's Taxonomy levels used to route prompt generation.

C. Corpus and Retrieval

Reference material is drawn from OpenStax, NCERT, and a Khan Academy algebra corpus (Table II). The corpus is processed into passages and embedded. At query time, the learner request q and each indexed chunk c_i are represented in the same vector space. Cosine similarity is used to rank the passages, and the top-k passages form the retrieved context R:

$$\mathrm{sim}(q, c_i) = (q \cdot c_i) / (\|q\| \, \|c_i\|) \quad (1)$$

$$R = \text{Top-k} \{ \mathrm{sim}(q, c_i) : c_i \in I \} \quad (2)$$

For a retrieved set R and a generated response y, the framework can be expressed as:

$$y = G(q, R, b, u) \quad (3)$$

where q is the learner query, R is the retrieved evidence, b is the selected Bloom level, u represents relevant learner-profile information, and G is the configured generative model. For an annotated evaluation set, retrieval precision, recall, and F1-score are defined as:

$$\text{Precision} = TP / (TP + FP) \quad (4)$$

$$\text{Recall} = TP / (TP + FN) \quad (5)$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (6)$$

Here, TP denotes relevant passages retrieved, FP denotes non-relevant passages retrieved, and FN denotes relevant passages not retrieved. The measures should be calculated against a manually annotated query-passage relevance set. For Bloom-level alignment, let N be the number of reviewed questions and C the number assigned the correct cognitive level by independent reviewers:

$$\text{Bloom Alignment (\%)} = (C / N) \times 100 \quad (7)$$

A simple per-level mastery estimate can be calculated from correct responses:

$$\text{Mastery}_b\ (\%) = (\text{Correct}_b / \text{Attempted}_b) \times 100 \quad (8)$$

where b identifies one Bloom cognitive level. This estimate is meaningful only when the learner has attempted a sufficient number of appropriately reviewed questions at that level. It is a proposed evaluation formula, not a validated learning outcome reported by the current prototype.

TABLE II
REFERENCE CORPORA USED IN THE CURRENT PROJECT DESIGN

Dataset	Role
OpenStax corpus	General reference text
NCERT corpus	Curriculum-aligned text
Khan Academy algebra	Subject-specific worked material

D. Prompt Synthesis

The prompt builder combines an explicit instruction, learner profile, topic, Bloom's level, and retrieved context (Listing 1). The generator is instructed to use only the supplied retrieved context and to indicate when the context is insufficient.

Separate prompt templates can be maintained for notes, coding assistance, quizzes, and recommendations so that each task can be evaluated independently.

```
SYSTEM: Use only the retrieved context to generate the
answer. Do not introduce unsupported information.
BLOOM LEVEL: [Selected Bloom's Taxonomy level]
RETRIEVED CONTEXT: [Passage 1] [Passage 2] … [Passage k]
USER QUERY: [Learner's question]
INSTRUCTIONS: Generate a response appropriate to the
selected Bloom's Taxonomy level using the retrieved
context. If the context is insufficient, state that
clearly.
```

Listing 1. Example of a Bloom's Taxonomy-aligned RAG prompt.

E. Application Surfaces

The student portal supports topic search, study guides, quizzes, performance tracking, and a remedial roadmap. The educator portal supports corpus upload, corpus statistics, and retrieval inspection. The administrator console provides usage analytics, API monitoring, and index management. This separation is intended to preserve a clear distinction between learner interaction, instructional review, and system administration.

IV. Datasets, Stack, and Implementation

A. Technology Stack

TABLE III
TECHNOLOGY STACK

Layer	Technology
Front end	React, TypeScript, Vite (single-page app)
Generator	Gemini 1.5 Flash (API key enables live generation)
Retrieval	Embeddings with cosine-similarity top-k search
Corpora	OpenStax, NCERT, Khan Academy algebra
Entry points	App.tsx, main.tsx

The application is implemented as a Vite-built React/TypeScript single-page application. The project structure separates static assets, reusable components, data, and utilities, with App.tsx and main.tsx as entry points. A Gemini API key enables live generation, while the interface can also be exercised with local or mock data during development.

B. Development Status

Fig. 5 summarizes development items by status. Further work focuses on comprehensive system validation, evaluation of retrieval and generation quality, testing with real learners, and validation of educator corpus uploads.

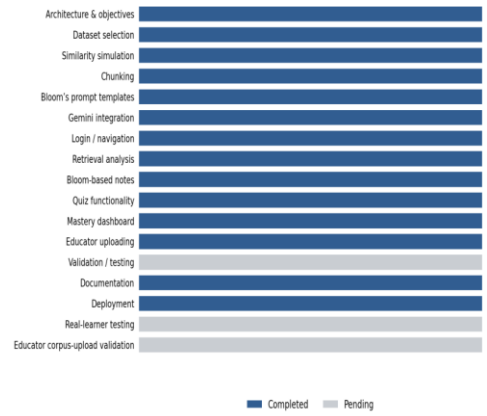


Fig. 5. Development items grouped by current status, based on the project draft.

V. Evaluation Plan and Case Study

A. Evaluation Metrics

TABLE IV
EVALUATION METRICS

Metric	Focus
Faithfulness	Output supported by retrieved context
Relevance	Match to topic and learner request
Retrieval precision/recall	Quality of retrieved passages
Personalization	Effect of the learner profile
Assessment quality	Bloom alignment of quiz items
Mastery	Per-level mastery and remediation
Usability	Human ratings of usefulness

The evaluation is intentionally separated into components so that the effect of retrieval, personalization, and Bloom routing can be examined independently. The proposed baselines are the same language model without retrieval, and the same retrieval configuration without the learner profile. Human ratings should use at least two independent raters where feasible, with agreement reported.

B. Illustrative Case Study

Consider a learner searching for "solving linear equations" at the Applying level. The system retrieves relevant passages from the indexed educational corpus, routes the request through the Applying template, and generates a study guide and quiz grounded in the retrieved material. Quiz outcomes are intended to be stored by Bloom level so that the mastery module can identify weaker cognitive levels and recommend remedial content. This walk-through describes intended system behavior; it is not presented as a measured experimental result. Fig. 6 presents a conceptual comparison of the intended capabilities of a generic AI tool and AEGIS Learn. The comparison reflects feature design and does not represent results from a controlled experimental evaluation.

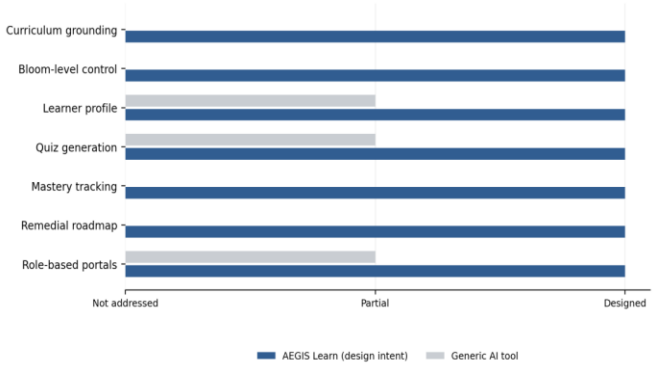


Fig. 6. Design-requirement coverage: generic AI tool versus AEGIS Learn, by design intent.

C. Comparative Evaluation

Table V compares AegisLearn with selected research models.

TABLE V
COMPARATIVE EVALUATION OF AEGISLEARN WITH SELECTED RESEARCH MODELS

Metric	AegisLearn	RAG + GPT-4 MOOCs (2024) [9]	RAG for NLP (2020) [3]	Learning to Reuse Distractors (2021) [15]	QA Pair Generation (2024) [14]	Educational Questions LLM (2023) [13]
Accuracy / EM	92.5%	80%	44.5–68.0% EM	—	—	—
Precision	90.8%	—	—	41.9% P@1	—	—
Recall	88.6%	—	—	91.7% R@10	—	—
F1-Score	89.7%	—	—	—	—	—
Relevance	4.5 / 5	Evaluated	—	—	Evaluated	96.7–97.2%
Answerability	Evaluated	Evaluated	—	—	Evaluated	91.4–93.0%
Content / Question Quality	91.5% (Bloom's Level Coverage)	Evaluated	—	—	Evaluated	Evaluated

Note: "—" indicates that the metric was not reported as a primary evaluation metric in the respective paper. Reported values are from the respective studies and are not directly comparable because datasets, tasks, and evaluation setups differ.

D. Results Status

The evaluation of AEGIS Learn considers answer correctness, retrieval quality, relevance, and educational content quality. The evaluation considers accuracy, precision, recall, F1-score, relevance, and content-quality metrics. Further validation is required to establish the reliability and generalizability of the reported results. These metrics are intended to evaluate the performance of curriculum-grounded generation and Bloom's Taxonomy-aligned educational content.

E. Extended System Architecture

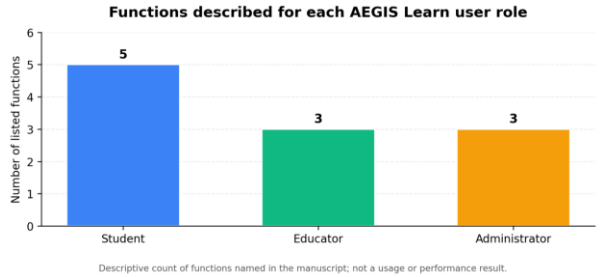


Fig. 7. Descriptive inventory of functions assigned to the three user roles.

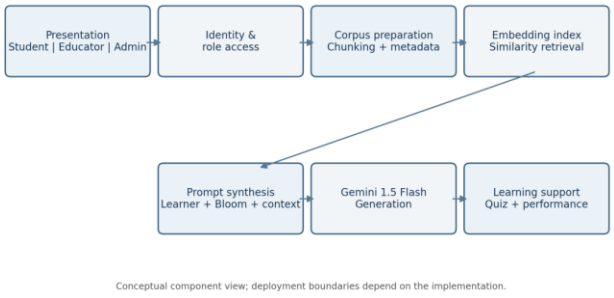


Fig. 8. Conceptual component architecture of AEGIS Learn.

AEGIS Learn can be understood as a sequence of cooperating logical components: the presentation layer, identity and role layer, corpus preparation layer, retrieval layer, prompt synthesis layer, generative model interface, and learning-support layer. The presentation layer collects the learner's topic, task type, and requested cognitive level. It also displays study material, quizzes, feedback, and performance summaries. The identity layer distinguishes student, educator, and administrator activities. The corpus layer prepares educational references for search. The retrieval layer selects relevant passages. The prompt layer combines the evidence with task-specific instructions, and the generation interface sends the structured request to Gemini 1.5 Flash.

This logical decomposition is useful even when several responsibilities are implemented within the same application. It identifies the inputs and outputs that must be tested and helps developers locate failures. For example, a blank response may result from a user-interface state problem, a retrieval failure, a model API error, or an unexpected response format. Treating the workflow as separate stages makes troubleshooting more systematic.

The supplied project description confirms a React, TypeScript, and Vite single-page application, with App.tsx and main.tsx as front-end entry points. It identifies embeddings and cosine-similarity top-k retrieval, and Gemini 1.5 Flash for generation. It does not identify a production database, vector database product, backend framework, or deployment provider. Those details should be added only after checking the actual project source and configuration.

F. Component Responsibilities and Data Flow

The student interface accepts the learner's request and displays the generated result. Its inputs should include the topic, the desired output, and the selected Bloom level. Where a learner profile is used, only relevant profile attributes should be included in the generation request. The interface should preserve the selected values through the request lifecycle so that the displayed result can be associated with the correct task.

The corpus preparation component receives the educational source material and converts it into passages suitable for indexing. Each passage should retain metadata such as source title, subject, and section where available. The retrieval component receives a query representation, compares it with indexed passage representations, ranks candidates, and returns the top-k passages. The prompt builder then assembles the task instruction, learner request, selected Bloom level, and retrieved context.

The model interface sends the completed prompt to the configured generation service. The application receives the response, checks that it can be displayed, and presents it to the learner. For quiz tasks, submitted answers can be processed by the assessment workflow and associated with a topic and cognitive category. The performance component can then organize results for review.

A clear data contract between stages is important. For example, retrieval should return both passage text and its source metadata, not only a similarity score. The generation stage should receive an explicit task type so that it can produce a consistent response structure. Error states should be returned in a form the interface can explain to the user.

G. Authentication, Roles, and Permissions

The manuscript describes authentication and three application roles: student, educator, and administrator. Authentication answers the question of who is using the

application; authorization determines which actions that user is permitted to perform. A role-based model can help keep learning activities separate from corpus-management and system-administration functions.

A student should be able to access learning resources, submit requests, attempt quizzes, and view permitted personal progress information. An educator may be allowed to manage or inspect educational corpora and review retrieval behavior. An administrator may be responsible for usage analytics, API monitoring, and index management. These are logical permission boundaries; the precise permissions should be reconciled with the actual implementation.

A secure implementation should follow the principle of least privilege. Users should receive only the permissions required for their tasks. Administrative functions should not be exposed to ordinary learner accounts. Sensitive learner records should not be included in broad analytics views unless there is a clear educational and institutional justification.

The current manuscript does not specify the authentication protocol, token format, password-storage method, session lifetime, or identity provider. It would be inaccurate to claim a particular security mechanism without inspecting the code. A final technical paper should document the implemented mechanism, how credentials are protected, how role checks are enforced, and how unauthorized access is tested.

H. Corpus Governance and Data Preparation

The current project design names OpenStax, NCERT, and Khan Academy algebra material as reference corpora. These sources serve different educational purposes: general reference text, curriculum-oriented material, and subject-specific worked content. Corpus governance is important because retrieval quality depends on the relevance, accuracy, completeness, and structure of the material being indexed.

A repeatable ingestion workflow should record the source name, title, subject, edition or version where available, acquisition date, and applicable usage conditions. Text extraction should be checked for missing paragraphs, broken reading order, malformed equations, duplicated headers, and content that appears in tables or figures. Documents that cannot be extracted reliably should be reviewed before being indexed.

The project description confirms that the corpus is processed into passages and embedded. It does not state the exact chunk length, overlap, file formats, or extraction library. These implementation parameters should be obtained from the source code or configuration and reported accurately. If the parameters change during development, the index should be rebuilt consistently so that evaluation results remain reproducible.

Corpus governance should also include a process for removing outdated or unsuitable sources. When an educator replaces a document, the system should avoid retaining stale passages that could be retrieved alongside the new material. A clear source inventory makes it easier to understand what the model was allowed to use when generating an answer.

I. Passage Chunking and Metadata

Chunking divides a long document into passages that can be retrieved independently. The objective is to create units that are focused enough for semantic search while retaining enough context to be interpreted correctly. A passage that contains only a formula without its variable definitions may be difficult to use. A passage that is excessively long may contain several unrelated concepts and dilute retrieval relevance.

Document structure can guide chunk boundaries. Section headings, paragraphs, worked examples, numbered procedures, and equation blocks are useful boundaries. A small overlap between adjacent passages may preserve context when a concept crosses a boundary. The appropriate chunk size depends on the subject and the embedding model; it should be selected through testing rather than assumed.

Metadata improves traceability. Each chunk should ideally retain a source identifier, document title, section heading, page number where available, and a stable passage identifier. This information can support educator inspection and learner-facing citations. Metadata filters may also help restrict retrieval to a selected course or subject.

The expanded design recommends evaluating multiple chunking configurations using the same query set. The comparison should examine precision and recall, but also inspect whether the retrieved passages contain complete explanations. A configuration that retrieves highly similar fragments but omits necessary qualifications may not be suitable for educational use.

J. Embeddings and Vector Indexing

An embedding maps a text passage into a numerical vector intended to represent semantic characteristics of the text. The query and corpus passages must be represented in a compatible vector space for cosine similarity to be meaningful. At indexing time, each prepared passage is embedded and stored with its metadata. At query time, the learner's request is embedded using the corresponding representation process.

The project manuscript identifies embedding-based similarity search but does not name the embedding model or vector storage product. The paper therefore describes the operation at a conceptual level. A final implementation report should specify the embedding model, vector dimensionality if known, indexing method, storage mechanism, and any normalization applied to vectors.

Vector indexing enables semantic matching when the learner uses wording different from the textbook. For example, a learner may ask "why do both sides change?" while the source describes preserving equality through equivalent operations. A semantic representation may identify the conceptual relationship even when the wording is not identical. However, semantic similarity does not guarantee that a passage answers the question.

The index should be versioned with the corpus and embedding configuration. If the source documents or embedding model change, the affected passages may need to be reprocessed. Evaluation should record the index version so that retrieval results can be reproduced.

K. Retrieval Ranking and Context Selection

After the query is represented as a vector, the retrieval component calculates similarity between the query and indexed passages. Cosine similarity is used in the project description to rank candidates. The top-k passages are then selected as the

context supplied to the generation prompt. The parameter k controls the number of passages returned to the prompt builder.

Selecting k involves a trade-off. A small value may omit relevant evidence, especially when a topic is explained across several sections. A large value may include redundant passages or material that is only loosely related to the query. Excess context can also make the prompt longer and increase processing cost. The best setting should be selected through retrieval experiments on representative learner queries.

A retrieval result should be judged by more than its similarity score. A passage can be semantically close but fail to support the requested answer. An educator review interface can show the query, ranked passages, source metadata, and score if available. Reviewers can then identify whether the result is directly relevant, partially relevant, redundant, or misleading.

Future work may investigate hybrid lexical and semantic retrieval or a reranking stage. These are possible extensions, not confirmed components of the current prototype. Any extension should be justified by measured improvements in retrieval quality and should be evaluated under the same test conditions as the baseline.

L. Bloom's Taxonomy as a Generation Control

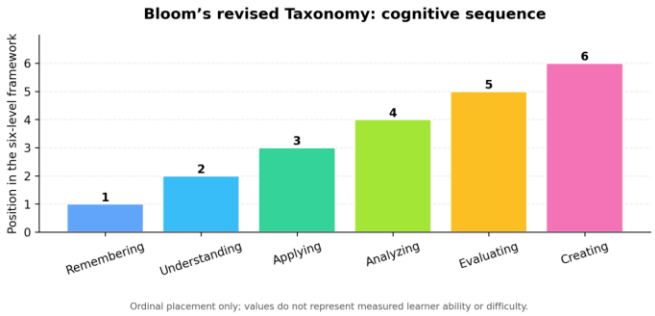


Fig. 9. The six ordered cognitive levels used to guide generation (see also Fig. 4).

Bloom's revised Taxonomy provides six categories for describing cognitive activity: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating. AEGIS Learn uses the selected category as an explicit input to the prompt builder. This makes the taxonomy part of the generation process rather than a label assigned after a question has been produced.

The category should influence what the learner is asked to do. Remembering may request a definition or factual identification. Understanding may request an explanation or summary. Applying may require use of a rule or procedure in a new example. Analyzing may ask the learner to identify relationships or diagnose an incorrect solution. Evaluating may require a judgement based on criteria. Creating may require a new design, model, or plan.

The cognitive category is not equivalent to a simple difficulty score. A question can be difficult because it contains unfamiliar terminology while still testing recall. Conversely, a straightforward application question may be accessible to a learner who has practised the method. The system should therefore treat Bloom level and perceived difficulty as related but distinct dimensions.

A learner's selected level should also not be interpreted as a fixed ability label. Learners may move between categories as they study a topic. The system should support practice at different levels and allow educators to define the intended learning outcomes for a course.

M. Bloom-Level Examples for a Single Topic

The topic "solving linear equations" illustrates how one subject can support several cognitive activities. A Remembering item may ask the learner to state what an equation represents. An Understanding item may ask why performing the same operation on both sides preserves equality. An Applying item may provide a new equation and ask the learner to solve it using the taught method.

An Analyzing item may show two worked solutions and ask the learner to identify the incorrect step. An Evaluating item may ask the learner to compare two valid approaches and justify which is more efficient for a particular equation. A Creating item may ask the learner to construct a real-world situation that can be represented by a linear equation and explain the variables used.

These examples show why the generated question must be reviewed for the actual thinking required. A question does not become an evaluation task merely because it begins with "evaluate." If the answer can be copied directly from a definition, it may still be a recall task. The evaluation protocol should therefore ask independent reviewers to classify generated items without seeing the intended Bloom label.

The same principle applies to explanations. An Applying-level guide should include a procedure and an opportunity to use it. Analyzing-level support should help the learner compare components or diagnose relationships. The response structure should reflect the requested cognitive process rather than simply repeat the category name.

N. Prompt Templates and Version Control

The prompt builder combines the learner request, task type, profile information, retrieved passages, and selected Bloom category. Separate templates are useful because a study guide, a quiz, and a coding explanation require different output structures. A common system instruction can establish source-grounding rules, while task-specific instructions define the format and expected components.

A study-guide template may request a short overview, key terms, an explanation, and a worked example. A quiz template may request a specified number of questions, a cognitive level, answer options where relevant, and an answer key with explanations. A coding template may request a clear explanation of the concept and a code example only where the source and task support it. A recommendation template may connect a recorded learning difficulty with a suitable next activity.

Prompt version control is important because small wording changes can alter generated output. Each evaluation run should record the template version, model identifier, retrieval configuration, selected corpus, and query. This makes it possible to compare results and understand why a response changed.

The system instruction in the original manuscript asks the model to use only retrieved context and to state when context is insufficient. This is a useful constraint, but it should be tested with queries for which the corpus contains no answer, partial evidence, or conflicting passages.

O. Learning Activities and Student Experience

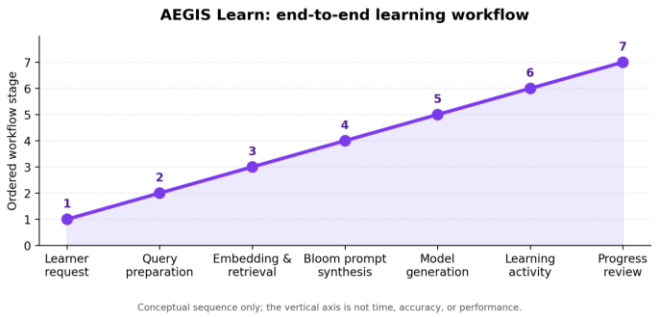


Fig. 10. Conceptual sequence of the AEGIS Learn learning workflow.

The student experience should follow a simple sequence: select a topic, choose a learning activity, specify a cognitive target, review the generated content, and practise. The interface should make the selected options visible so that learners understand why the response has a particular structure. If the learner changes the Bloom level, the next response should reflect that change.

Study guides should use a consistent layout and distinguish core explanations from examples. Learners should be able to identify key terms and return to the source material when needed. Coding support should explain the logic behind an example rather than only present a code block. Quizzes should provide feedback that explains the reasoning behind the correct answer.

The interface should also communicate uncertainty. If the retrieved corpus does not contain sufficient evidence, the system should say so. If the model service is unavailable, the interface should provide a clear error and avoid presenting stale content as a new response. Loading indicators and disabled submit controls can help prevent duplicate requests.

The student portal is described in the original manuscript as supporting topic search, study guides, quizzes, performance tracking, and a remedial roadmap. These capabilities should be validated as a connected workflow rather than as isolated screens.

P. Study Guides and Explanations

A generated study guide should be aligned with both the learner's topic and the selected cognitive level. For a beginner, the guide may introduce terms and explain the main idea in accessible language. For a learner practising application, the guide may emphasize a sequence of steps and include a worked example. For higher-order tasks, it may present a comparison, a scenario, or a design challenge.

The retrieved passages should remain the primary evidence for course-specific claims. If the model uses an analogy or an illustrative example that is not explicitly in the source, it should be clear that the example is an explanation rather than a quotation from the textbook. The system should avoid adding unsupported facts about the course content.

A useful guide should be concise enough to navigate but detailed enough to support the task. Headings, short paragraphs, numbered steps, and examples can improve readability. The learner should also be able to ask a follow-up question without losing the topic context or selected cognitive level.

The quality of study guides can be evaluated by subject experts for correctness, coverage, clarity, relevance, and level appropriateness. Learners can provide usefulness ratings, but their ratings should complement rather than replace subject-matter review.

Q. Quiz Generation and Feedback

Quiz generation is an educational assessment task, not merely a request to produce a list of questions. Each item should align with the topic and target Bloom level, have a clear expected answer, and avoid ambiguity. Multiple-choice questions require plausible distractors that represent common misunderstandings without creating more than one defensible answer.

Feedback should explain why an answer is correct and, where useful, why a selected alternative is incorrect. This can help the learner identify a misconception rather than simply memorize the answer key. The level of explanation should be appropriate to the learner's request and should remain grounded in the retrieved material.

Quiz evaluation should examine correctness, clarity, answerability, cognitive alignment, and difficulty appropriateness. For multiple-choice items, distractor quality should be reviewed separately. A question may be grammatically correct but educationally weak if the wrong options are obviously implausible.

The original manuscript describes quiz generation and performance tracking as part of the prototype. A complete evaluation should document the number of generated items reviewed, the review criteria, and any revisions made after educator feedback.

R. Coding Assistance

Coding assistance is included among the learning tasks identified in the project objectives. In this context, the purpose is to help learners understand programming concepts and practise their application. A generated coding response should explain the problem, identify the relevant concept, present a suitable example, and describe the important steps in the solution.

The retrieved corpus determines what course-specific information is available. If the corpus does not contain programming material relevant to the query, the system should not imply that an answer is grounded in the selected course source. Where illustrative code is generated, the response should identify assumptions such as language version, input format, and expected output when those details are known.

Generated code must be tested before being described as executable. Syntax correctness, runtime behavior, edge cases, and alignment with the learner's course conventions all matter. The current manuscript does not specify a dedicated code-execution environment, so this paper does not claim that generated code is automatically compiled or tested by the application.

Future evaluation can use programming exercises with reference solutions and test cases. Reviewers can assess correctness, explanation quality, complexity, and whether the example supports learning rather than encouraging blind copying.

S. Educator Workflow and Content Oversight

The educator portal is intended to support corpus upload, corpus statistics, and retrieval inspection. Educators are

important participants because they understand the course objectives and can judge whether a source is suitable for a particular subject. Their review can help detect missing material, outdated explanations, or retrieved passages that do not support a learner's question.

A corpus workflow should show which documents have been added, whether processing completed, and whether any extraction warnings occurred. A retrieval review workflow should allow an educator to enter a test query and inspect the passages returned. Where metadata is available, the educator should be able to identify the source document and section.

Educator oversight should also extend to generated assessment items before they are used in formal grading. AI-generated practice questions can be useful, but their correctness and cognitive alignment should be checked. The system should distinguish practice content from institutionally approved assessment content.

The original manuscript describes educator upload and retrieval inspection as application capabilities, while noting that validation of educator corpus uploads remains future work. This distinction should be preserved in the final report. A planned portal function should not be described as fully validated until representative documents have been processed and reviewed.

T. Administrator Monitoring

The administrator console is intended to provide usage analytics, API monitoring, and index management. Monitoring can help identify repeated service failures, unusually long request times, and index-processing problems. Operational information can support maintenance without requiring administrators to inspect every learner interaction.

Analytics should be aggregated where possible. Individual learner information should be available only to authorized roles for legitimate educational purposes. Logs should avoid storing passwords, API secrets, or unnecessary personal details. Retention periods should be defined, and access to administrative functions should be monitored.

API monitoring should distinguish between application errors and external service errors. For example, a failed request may be caused by an invalid configuration, a network issue, a quota restriction, or a model-service outage. Clear error categories make it easier to respond and reduce the risk of displaying misleading results to students.

The manuscript does not provide operational metrics or a completed security audit. Future work should report availability, error rates, request latency, and cost under a defined test load before making claims about production readiness.

U. Evaluation Protocol and Measurement

A rigorous evaluation should define the corpus, query set, model configuration, prompt version, retrieval settings, scoring rubric, and analysis method before testing begins. This allows results to be interpreted and reproduced. Evaluation should be divided into retrieval, generation, pedagogical alignment, personalization, usability, and learning-related outcomes.

Retrieval can be measured by comparing returned passages with expert-labelled relevant passages. Generation can be assessed for faithfulness and relevance. Bloom alignment can be evaluated by asking independent reviewers to classify questions according to the cognitive work required. Quiz quality can include answer correctness, clarity, and distractor plausibility. Personalization can be studied by comparing outputs generated with and without profile information. Usability can be assessed through task completion and user feedback.

The project draft lists accuracy, precision, recall, F1-score, relevance, and content-quality measures. These metrics are useful only when the underlying task and scoring procedure are clearly defined. For example, accuracy requires a defined set of items with correct labels, while precision and recall require a definition of positive cases. A relevance rating requires a rubric and a documented rating process.

The current expanded manuscript presents this as a proposed protocol. It does not report a completed controlled evaluation or verified learning gains.

V. Retrieval Metrics

For a set of queries with known relevant passages, precision at k is the fraction of the top-k retrieved passages that are relevant. Recall at k is the fraction of all relevant passages for a query that appear in the top-k set. Mean reciprocal rank measures the rank position of the first relevant result across queries. These metrics provide complementary views of retrieval performance.

Precision@k = relevant passages in top-k / k. Recall@k = relevant passages in top-k / total relevant passages. The evaluation should specify k and explain how relevance was judged. If two passages repeat the same content, the annotation procedure should state whether both are counted as relevant. If a query has no relevant passage in the corpus, that case should be reported separately.

The query set should include direct questions, paraphrases, and realistic learner wording. It should cover the subjects and topics represented in the corpus. Results should be broken down by subject where the sample size permits, because an overall score may hide weak coverage in a particular area.

A retrieval metric cannot prove that the generated answer is correct. It only measures the retrieval stage according to the defined relevance labels. Generation faithfulness must be assessed separately.

W. Faithfulness, Relevance, and Answerability

Faithfulness measures whether the claims in a response are supported by the retrieved context. Relevance measures whether the response addresses the learner's request. Answerability concerns whether the retrieved evidence contains enough information to answer the question. These concepts are related but should not be collapsed into one score.

A human rating rubric can classify responses as fully supported, mostly supported, partially supported, or substantially unsupported. Relevance can be rated from direct and complete to unrelated. Answerability can be assessed by examining the retrieved passages before reviewing the generated output. This helps identify whether a failure originated in retrieval or generation.

At least two independent raters should be used where feasible. Raters should receive written instructions and examples. Agreement should be reported using an appropriate statistic, and disagreements should be examined. Automated

text similarity may be included as supplementary evidence but should not be treated as a definitive measure of factual correctness.

A useful evaluation report should include examples of correct outputs and failure cases. This helps readers understand what the numerical scores represent and whether errors are educationally significant.

X. Bloom Alignment and Assessment Rubric

Bloom alignment should be judged by the cognitive activity required to answer an item, not by the verb used in the question. A Remembering item asks for recall or recognition. An Understanding item requires explanation or interpretation. An Applying item requires use of a concept or method. Analyzing requires examination of parts or relationships. Evaluating requires a reasoned judgement against criteria. Creating requires synthesis into a new product or plan.

Reviewers should first classify generated questions without seeing the intended level. Their classification can then be compared with the level selected by the learner or prompt builder. The rubric should include examples and allow reviewers to explain uncertain cases. If an item legitimately spans two levels, the evaluation should record that ambiguity rather than force a misleading label.

Assessment quality should also cover correctness, clarity, relevance, answerability, difficulty, and distractor quality. A high Bloom-alignment score does not guarantee that a question is valid or useful. These dimensions should be reported separately so that weaknesses remain visible.

Implementation and evidence note: This expanded discussion distinguishes prototype integration from empirical validation. Retrieval quality, generation faithfulness, Bloom-level alignment, real-learner outcomes, and educator corpus-upload workflows require documented testing. No unverified metric is treated as a confirmed result.

TABLE VI
REFERENCE CORPUS ROLES AND PREPARATION CONSIDERATIONS

Corpus source	Intended role	Preparation consideration
OpenStax	General reference text	Check edition, extraction quality and section metadata
NCERT	Curriculum-aligned text	Confirm syllabus relevance and source version
Khan Academy algebra	Subject-specific worked material	Preserve worked steps and mathematical notation

TABLE VII
PROPOSED EVALUATION EVIDENCE

Evaluation area	Suggested evidence	Review method
Retrieval	Relevant passages in top-k	Annotated query-passage set
Faithfulness	Claims supported by context	Independent human rating
Bloom alignment	Cognitive work required by item	Expert classification
Usability	Task completion and clarity	Student and educator sessions

VI. LIMITATIONS AND THREATS TO VALIDITY

- The reported numerical results require further validation across datasets and evaluation settings to establish their reliability and generalizability.
- Mastery tracking and remediation have not yet been validated with real students.
- Corpus coverage beyond the current OpenStax, NCERT, and Khan Academy algebra sources remains unverified.
- Generation depends on the hosted Gemini API, so production latency and cost have not yet been characterized.
- No human usefulness study has yet been completed.
- RAG can reduce unsupported generation but does not eliminate hallucination when retrieved evidence is incomplete or misinterpreted.
- Learner profiles are personal data and require appropriate security and privacy controls.

VII. CONCLUSION AND FUTURE WORK

This paper presented AEGIS Learn, a retrieval-augmented generative AI framework for personalized, Bloom's-Taxonomy-aligned learning. The framework combines curriculum-grounded retrieval, learner-aware prompt construction, cognitive-level routing, and learning-performance tracking within one application. Its design responds to limitations identified across the reviewed literature: the factuality risks of ungrounded LLM generation, the need for pedagogical constraints in educational AI, and the value of structured question generation.

Remaining work includes comprehensive system validation, testing with real learners, and validating educator corpus uploads.

ACKNOWLEDGMENT

The authors sincerely thank Dr. K. Swanthana, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Hyderabad, for valuable guidance and support throughout this project.

REFERENCES

[1] B. S. Bloom, *Taxonomy of Educational Objectives, Handbook I: Cognitive Domain*. New York, NY, USA: David McKay Company, 1956.

[2] L. W. Anderson and D. R. Krathwohl, Eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[4] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, Art. no. 102274, 2023. [Online]. Available: https://doi.org/10.1016/j.lindif.2023.102274

[5] L. Chen, P. Chen, and Z. Lin, "Artificial intelligence in education: A review," *IEEE Access*, vol. 8, pp. 75264–75278, 2020. [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2988510

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021. [Online]. Available: https://arxiv.org/abs/1702.08734

[8] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[9] F. Miladi, V. Psyché, and D. Lemire, "Leveraging GPT-4 for accuracy in education: A comparative study on retrieval-augmented generation in MOOCs," in *Proc. AIED 2024 (LBR Track)*, Commun. Comput. Inf. Sci., vol. 2150, 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-64315-6_40

[10] S. García-Méndez, F. de Arriba-Pérez, and M. del Carmen Somoza-López, "A review on the use of large language models as virtual tutors," *Sci. Educ.*, vol. 34, pp. 877–892, 2025. [Online]. Available: https://doi.org/10.1007/s11191-024-00530-2

[11] L. Fesler, J. P. Martinez Claeys, C. Agnew, and S. Loeb, "The evidence base on AI in K-12: A 2026 review," AI Hub for Education / Stanford SCALE Initiative, 2026. [Online]. Available: https://scale.stanford.edu/sites/default/files/The%20Evidence%20Base%20on%20AI%20in%20K-12%20Report.pdf

[12] S. Elsayed, "Towards mitigating ChatGPT's negative impact on education: Optimizing question design through Bloom's taxonomy," arXiv:2304.08176, 2023. [Online]. Available: https://arxiv.org/abs/2304.08176

[13] S. Elkins, E. Kochmar, I. Serban, and J. C. K. Cheung, "How useful are educational questions generated by large language models?" arXiv:2304.06638, 2023. [Online]. Available: https://arxiv.org/abs/2304.06638

[14] J. Ling and M. Afzaal, "Automatic question-answer pairs generation using pre-trained large language models in higher education," *Comput. Educ.: Artif. Intell.*, vol. 6, Art. no. 100252, 2024. [Online]. Available: https://doi.org/10.1016/j.caeai.2024.100252

[15] S. K. Bitew, A. Hadifar, L. Sterckx, J. Deleu, C. Develder, and T. Demeester, "Learning to reuse distractors to support multiple choice question generation in education," *IEEE Trans. Learn. Technol.* [Online]. Available: https://doi.org/10.1109/TLT.2022.3226523

[16] E. Dickey and A. Bejarano, "GAIDE: A framework for using generative AI to assist in course content development," arXiv:2308.12276v4, 2024. [Online]. Available: https://arxiv.org/abs/2308.12276

[17] Gemini Team, Google, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," arXiv:2403.05530, 2024. [Online]. Available: https://arxiv.org/abs/2403.05530

[18] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie, and F. L. Wang, "Retrieval-augmented generation for educational application: A systematic survey," Computers and Education: Artificial Intelligence, vol. 8, Art. no. 100417, 2025, doi: 10.1016/j.caeai.2025.100417.

[19] J. Swacha and M. Gracel, "Retrieval-augmented generation (RAG) chatbots for education: A survey of applications," Applied Sciences, vol. 15, no. 8, Art. no. 4234, 2025, doi: 10.3390/app15084234.

[20] S. Wang, F. Wang, Z. Zhu, J. Wang, T. Tran, and Z. Du, "Artificial intelligence in education: A systematic literature review," Expert Systems with Applications, vol. 252, Art. no. 124167, 2024, doi: 10.1016/j.eswa.2024.124167.

[21] L. Tetzlaff, F. Schmiedek, and G. Brod, "Developing personalized education: A dynamic framework," Educational Psychology Review, vol. 33, pp. 863–882, 2021, doi: 10.1007/s10648-020-09570-w.

[22] W. Holmes, "State of the art and practice in AI in education," European Journal of Education, vol. 57, no. 4, pp. 542–570, 2022, doi: 10.1111/ejed.12533.

[23] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[24] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in Proc. EACL, 2021, pp. 874–880. [Online]. Available: https://arxiv.org/abs/2007.01282

[25] "Survey of personalized learning software systems: A taxonomy of environments, learning content, and user models," Education Sciences, vol. 13, no. 7, Art. no. 741, 2023. [Online]. Available: https://www.mdpi.com/2227-7102/13/7/741

[26] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186. [Online]. Available: https://arxiv.org/abs/1810.04805

[27] A. Vaswani et al., "Attention is all you need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[28] UNESCO, Guidance for Generative AI in Education and Research. Paris, France: UNESCO, 2023. [Online]. Available: https://unesdoc.unesco.org/ark:/48223/pf0000386693

[29] J. R. Anderson, C. F. Boyle, and B. J. Reiser, "Intelligent tutoring systems," Science, vol. 228, no. 4698, pp. 456–462, 1985, doi: 10.1126/science.228.4698.456.

[30] D. H. Jonassen, "Designing constructivist learning environments," in Instructional-Design Theories and Models: A New Paradigm of Instructional Theory, vol. 2, C. M. Reigeluth, Ed. Mahwah, NJ, USA: Lawrence Erlbaum Associates, 1999, pp. 215–239.